

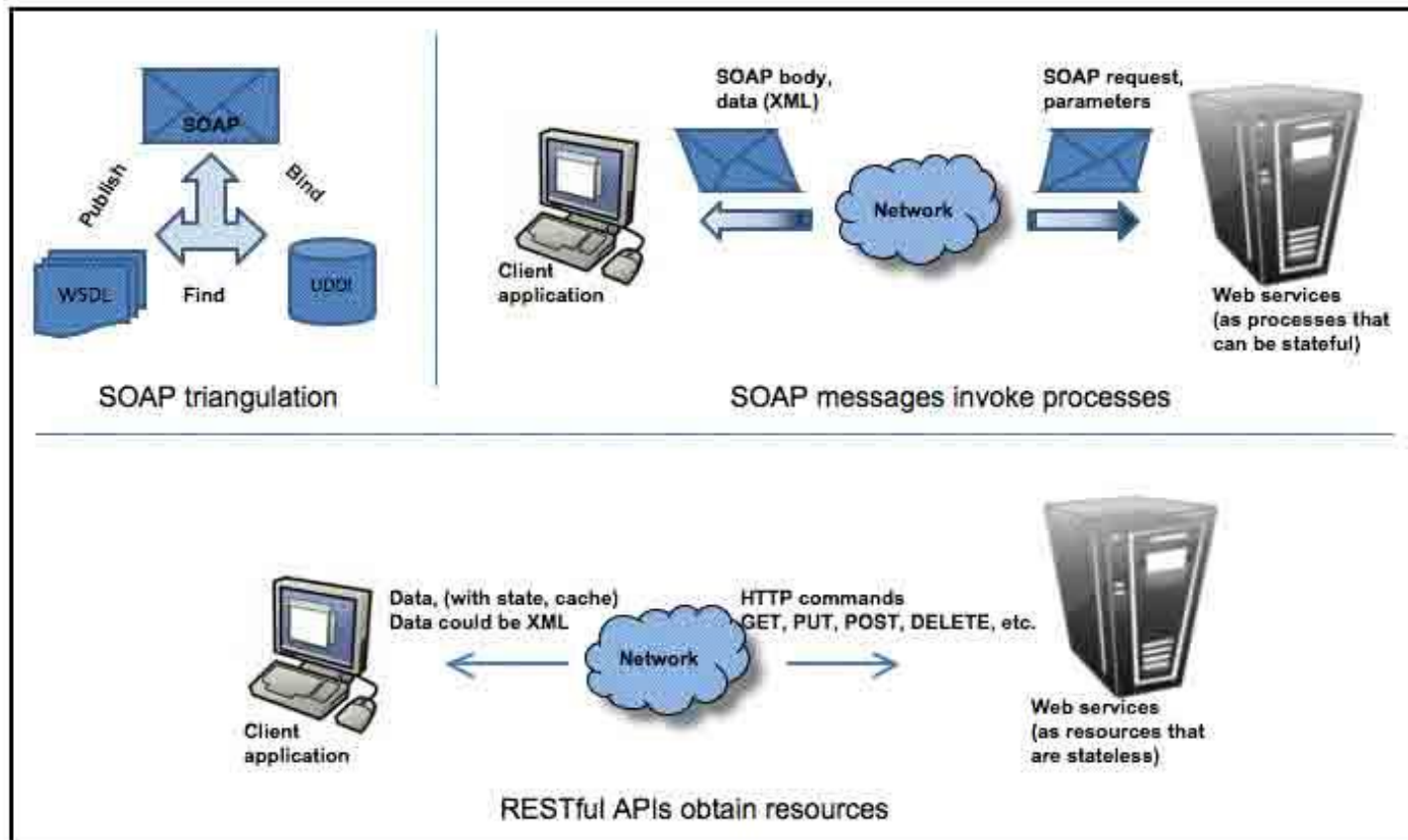
Développement Web Services

Web Services

SOAP (JAX-WS)

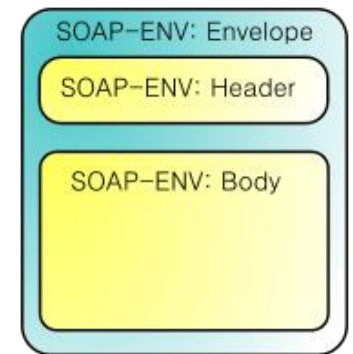
Les Services Web

(SOAP vs REST)



Les services web SOAP

- **SOAP** signifie : **Simple Object Access Protocol**
- Utilise le protocole **RPC** (Remote Procedure Call) et basé sur **XML**
- Autorise la transmission de messages entre deux objets distants
- Habituellement utilisé avec HTTP mais peut être utilisé avec d'autres protocoles (SMTP ou FTP)
- Les messages SOAP sont composés de deux parties:
 - Un **en-tête** (optionnel)
 - Un **corps**
- Les deux parties sont à l'intérieur d'une **enveloppe SOAP**



Exemple d'un message SOAP

Signature de la méthode

```
int doubleAnInteger ( int numberToDouble );
```

Requête SOAP

```
<?xml version="1.0" encoding="UTF-8" standalone="no" ?>
<SOAP-ENV:Envelope xmlns: ... >
  <SOAP-ENV:Body>
    <ns1:doubleAnInteger xmlns:ns1="urn:MySoapServices">
      <param1 xsi:type="xsd:int">123</param1>
    </ns1:doubleAnInteger>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

Exemple d'un message SOAP

Réponse SOAP

```
<?xml version="1.0" encoding="UTF-8" standalone="no" ?>
<SOAP-ENV:Envelope
  xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/"
  xmlns:xsd="http://www.w3.org/1999/XMLSchema">
  <SOAP-ENV:Body>
    <ns1:doubleAnIntegerResponse
      xmlns:ns1="urn:MySoapServices"
      SOAP-ENV:encodingStyle=
        "http://schemas.xmlsoap.org/soap/encoding/">
      <return xsi:type="xsd:int">246</return>
    </ns1:doubleAnIntegerResponse>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

Exemple d'un message SOAP

Les entêtes SOAP n'ont pas été ajoutés aux messages précédents (ils sont optionnels)

```
<soap:Envelope xmlns:soap="...">
  <soap:Header>
    <UsernameToken xmlns="http://abc.com/webservices">
      user
    </UsernameToken>
    <PasswordText xmlns="http://abc.com/webservices">
      hello123
    </PasswordText>
  </soap:Header>
  <soap:Body><!-- data goes here --></soap:Body>
</soap:Envelope>
```

WSDL

- Comment une application cliente peut-elle connaître la procédure (service) qu'elle peut appeler?
- C'est grâce au **WSDL** (Web Services Description Language)

"Web Services Description Language is an XML format for describing network services as a set of endpoints operating on messages containing either document-oriented or procedure-oriented information."

WSDL

La structure du WSDL

```
<definitions>

  <types>
    <!-- The data types used by the web service -->
  </types>

  <message>
    <!-- The messages used by the web service -->
  </message>

  <portType>
    <!-- The operations performed by the web service -->
  </portType>

  <binding>
    <!-- The communication protocols used by the web service -->
  </binding>

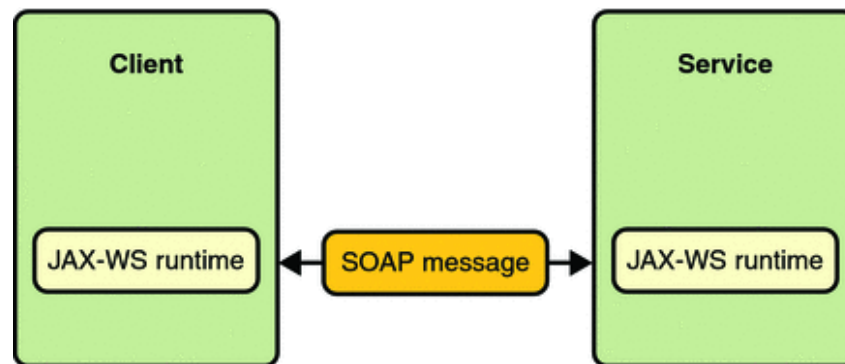
</definitions>
```


WSDL

- **Port Type WSDL**
Décrit un service Web, les opérations pouvant être effectuées et les messages impliqués
- **Messages WSDL**
Définit les éléments de données d'une opération
- **Types WSDL**
Définit les types de données utilisés par le service Web
- **Bindings WSDL**
Définit le format du message et les détails du protocole pour chaque port

Les services web SOAP avec JAX-WS

- **JAX-WS** est une API qui peut être utilisée:
 - Du côté serveur pour définir les services Web
 - Du côté client pour consommer des services Web



- Les services Web SOAP sont **interopérables** :

Les services web SOAP avec JAX-WS

Développement d'un service web SOAP avec JAX-WS

Pour définir un service web SOAP, on utilise l'annotation **@WebService**

```
@WebService
public class CircleFunctions {

    public double getArea(double r) {
        return java.lang.Math.PI * (r * r);
    }

    public double getCircumference(double r) {
        return 2 * java.lang.Math.PI * r;
    }

}
```

Les services web SOAP avec JAX-WS

Développement d'un service web SOAP avec JAX-WS

L'annotation **@WebService** peut avoir certains attributs:

```
@WebService (  
    name="MyWebService",  
    serviceName="UsefulService",  
    targetNamespace="http://ws.site.com/sample"  
)  
public class MyWebService {  
    ...  
}
```

Les services web SOAP avec JAX-WS

Développement d'un service web SOAP avec JAX-WS

JAX-WS fournit également une annotation **@WebMethod** pour personnaliser les méthodes du service Web:

```
@WebMethod(operationName="computeArea")  
public double getArea(double r) {  
    return java.lang.Math.PI * (r * r);  
}
```

Cette annotation peut avoir plusieurs attributs:

- **operationName**: Nom du wsdl: opération correspondant à cette méthode
- **exclude**: Si true, marque cette méthode pour qu'elle ne soit pas exposée (prend la valeur false par défaut)

Les services web SOAP avec JAX-WS

Développement d'un service web SOAP avec JAX-WS

JAX-WS fournit aussi une annotation **@WebParam** pour personnaliser les paramètres du service Web:

```
@WebMethod(operationName="computeArea")
public double getArea(@WebParam(name = "r") double r) {
    return java.lang.Math.PI * (r * r);
}
```

Cette annotation peut avoir plusieurs attributs tels que : **header**, **name**, etc.

Les services web SOAP avec JAX-WS

Développement d'un service web SOAP avec JAX-WS

JAX-WS prend en charge d'autres annotations pour personnaliser et contrôler encore plus le résultat du service Web:

@WebResult

@SOAPBinding

@BindingType

Les services web SOAP avec JAX-WS

Génération du WSDL

Mais comment générer le fichier WSDL basé sur ces services ?

- JAX-WS le fait pour nous !

Disponible sur cette URL:



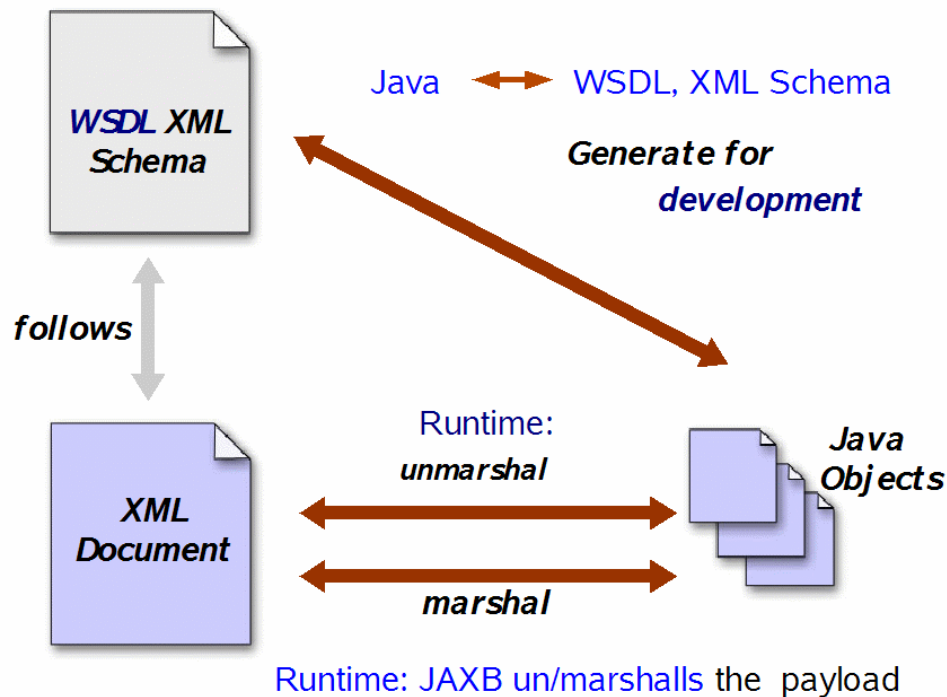
http: // <hôte> / <chemin du contexte> / <nom du service>? WSDL

Les services web SOAP avec JAX-WS

JAX-WS côté serveur

Lorsque l'application sera déployée, ces fichiers seront générés sur le serveur:

- Le **WSDL** (à partir des classes des services web)
- Des **fichiers XML** correspondant aux classes Entités (XML data-binding (grâce à l'API JAXB))



Les services web SOAP avec JAX-WS

Déploiement

Le serveur d'application Glassfish permet de voir et de tester les méthodes du services Web en offrant un client Web prêt à l'emploi.

Il suffit d'entrer simplement l'URL du service Web suivi de **?Tester**

Par exemple:

`http://localhost:8080/MyWebService/Calculator?Tester`

TP 4 - SOAP

Travail à réaliser (1/2)

I) Production du web service :

- 1) Créer un projet Java "TP4-SOAPWS"
 - 2) Créer la classe "Compte" (dans un package nommé "metier")
Attributs : int code / double solde / Date dateCreation (+ Getters & Setters + Constructeurs)
 - 3) Créer la classe "BanqueService" (dans un package nommé "service")
Créer 3 méthodes : double conversion(double mt) / Compte consulterCompte(int code) /
List<Compte> listComptes()
Ajouter les annotations nécessaires pour transformer ce POJO en service web SOAP
 - 4) Créer une classe main "ServeurJaxWS" (default package)
créer une chaîne de caractères nommée url = "http://localhost:8585/"
créer le WS Endpoint : Endpoint.publish(url , new BanqueService())
Exécuter le projet et se connecter à un navigateur afin d'afficher le fichier WSDL de notre WS.
(http://localhost:8585/BanqueWS?wsdl)
Visualiser le contenu de ce fichier et le comparer avec la structure vue au cours.
- Remarque : Dans le cas où on crée un projet web JEE qui produit le web service,
il suffit de déployer le projet dans un serveur web (ex: Tomcat) ou un serveur d'application (ex: Glassfish) afin de lancer le web service.

TP 4 - SOAP

Travail à réaliser (2/2)

II) Consommation du Web service :

- 1) Créer un projet Java "TP4-SOAPWSCClient"
- 2) Créer un web service client (new -> Other -> Web services -> Web service client)

Ajouter le lien du wsdl dans le champ "Service Definition"

(un ensemble de classes vont être générée dans le projet)

- 3) Créer une classe main "Client"
- 4) Ajouter la méthode : `private static double convert(double m)`

Ajouter le code suivant :

```
BanqueServiceServiceLocator service = new BanqueServiceServiceLocator();  
BanqueWS port = service.getBanqueWSPort();
```

Appeler la méthode "ConversionDhToEuro(int m)" via : `return port.conversionDhToEuro(m) ;`

- 5) Appeler dans le main la méthode `convert(10)` et afficher le résultat puis exécuter la classe "Client".
- 6) Ajouter deux autres méthodes pour appeler les deux autres services restants.